

ГОВОР ЗА ОДБРАНУ ДИПЛОМСКОГ РАДА

Имплементација обсервабилности микросервисне архитектуре на примеру TravelPlanner апликације

Укупно циљано трајање излагања: ~15 минута (900 секунди) • 16 слајдова

СЛАЈД 1 — Насловни слајд

Оквирно време: 0:20 минута

Добар дан/вече, поштована комисијо. Ја сам Дајана Булић, а тема мог завршног рада је „Имплементација обсервабилности микросервисне архитектуре на примеру TravelPlanner апликације“, под менторством др Себастијана Стоје.

СЛАЈД 2 — Travel Planner — постојећа апликација

Оквирно време: 0:55 минута

Travel Planner је постојећа микросервисна веб апликација за планирање путовања. Корисник на једном месту може да организује цео план путовања — дестинације, дневне активности кроз календар, трошкове и буџет, checklist и подсетнике. План може да подели и са другим особама путем кода или QR кода.

Систем је реализован кроз четири одвојена сервиса на Microsoft Service Fabric платформи: Gateway, AuthService, TripService и SharingService.

СЛАЈД 3 — Микросервиси су тешки за праћење

Оквирно време: 0:55 минута

Микросервисна архитектура има предности као што су независно скалирање и одвојен развој сервиса. Међутим, доноси и један озбиљан проблем: један кориснички захтев може да прође кроз више процеса, база и мрежних позива.

Када нешто пође наопако, питања као што су „зашто је овај захтев спор“, „где тачно долази до грешке“ или „да ли је систем тренутно здрав“ постају много тежа.

Управо на та питања постојећи Travel Planner није могао да да одговор.

СЛАЈД 4 — Циљ рада

Оквирно време: 0:40 минута

Циљ рада био је да се, пре свега, анализира постојећи систем са аспекта обсервабилности и идентификују конкретни недостаци.

Након тога, циљ је био да се систем надогради слојем обсервабилности заснованим на OpenTelemetry стандарду. При томе је било важно да се не мења постојећа пословна логика.

На крају, било је потребно и доказати да надоградња заиста функционише — кроз извршавање система, аутоматизоване тестове и мерљиве резултате, а не само кроз инсталацију алата.

СЛАЈД 5 — Архитектура пре надоградње (AS-IS)

Оквирно време: 1:10 минута

Овде је приказана архитектура система пре надоградње.

Gateway, AuthService и TripService су stateless сервиси, што значи да не чувају стање између позива и ослањају се на своје базе. SharingService је stateful зато што се важећи кодови за дељење кеширају на Service Fabric чвору кроз Reliable Collections, како би били доступни са ниском латенцијом.

Свака компонента има своју базу, по принципу database-per-service.

На дијаграму је најважнији основни ток: Frontend → Gateway → остали сервиси → базе.

СЛАЈД 6 — Идентификовани недостаци

Оквирно време: 1:10 минута

Детаљном анализом дошла сам до пет конкретних недостатака.

Први је немогућност праћења захтева кроз више сервиса, јер није постојао trace ID.

Други је одсуство метрика перформанси, као што су CPU, меморија, латенција и стопа грешака.

Трећи су „тихе“ грешке. Пронађен је конкретан пример у коду где се грешка ухвати, али се нигде не логује, па не остаје траг о томе шта се догодило.

Четврти недостатак је то што сервиси нису имали health check механизам.

И пети је минимално логовање — претрагом backend кода нисам пронашла ниједан позив ILogger пре ове надоградње.

СЛАЈД 7 — OpenTelemetry + Grafana stack

Оквирно време: 1:20 минута

Уместо да сваки од ових проблема решавам засебно, изабрала сам OpenTelemetry као централни, vendor-неутралан стандард за traces, metrics и logs.

OpenTelemetry има готову инструментацију за ASP.NET Core, HttpClient и SqlClient. Сервиси шаљу телеметрију ка OpenTelemetry Collector-у, који је даље усмерава ка Prometheus-у за метрике, Grafana Tempo-у за дистрибуирано праћење захтева и Grafana Loki-ју за логове.

Grafana служи као јединствени визуелни слој, са могућношћу да се из лога пређе директно на trace, а из trace-а на метрику.

СЛАЈД 8 — TO-BE архитектура

Оквирно време: 1:10 минута

Ово је циљна архитектура након надоградње. У односу на претходни дијаграм, додат је observability слој који чине Collector, Prometheus, Tempo, Loki и Grafana.

Тај слој ради кроз Docker Compose и независан је од постојећег Service Fabric кластера. На тај начин није било потребно мењати постојећу пословну логику — додата је само инструментација.

Такође, за пропагацију контекста није било потребно писати custom код.

СЛАЈД 9 — Health checks

Оквирно време: 0:55 минута

Сваки сервис сада излаже два endpoint-а: /health/live и /health/ready.

Live endpoint показује да ли процес уопште ради, док ready endpoint проверава да ли су критичне зависности спремне, на пример да ли је SQL база доступна.

Gateway нема сопствену базу, па он проверава доступност три downstream сервиса, користећи исте адресе које већ користи YARP за рутирање.

Одговор је структуриран у JSON формату, а на примеру се види стање у ком су сва три сервиса Healthy.

СЛАЈД 10 — Grafana dashboard — све на једном месту

Оквирно време: 0:55 минута

Овај dashboard приказује RED метрике — request rate, error rate и p95 latency — као и ресурсе попут CPU-а, меморије и garbage collector-а.

Ту је и ток логова, који може да се филтрира по сервису.

На овај начин су решени недостаци везани за праћење перформанси и логовање, јер више није потребно ручно прегледати конзолу да би се добила слика о стању система.

СЛАЈД 11 — Тестирање — стратегија

Оквирно време: 1:05 минута

Да dashboard не би био само визуелни доказ, урађено је и аутоматизовано тестирање.

Направљена су четири xUnit пројекта, по један за сваки сервис. Сваки тест подиже прави Kestrel host на ефемерном порту, без mock-а базе података.

HealthEndpointsTests проверавају health check endpoint-е, ObservabilityPipelineTests проверавају да ли се OTel инструментација исправно активира, а TripClientTests покривају понашање везано за „тихи“ catch блок.

Укупно, свих 21 тест пролази.

СЛАЈД 12 — Доказ да тестови стварно нешто проверавају

Оквирно време: 0:55 минута

Да бих показала да тестови нису само формалност, намерно сам привремено вратила оригинални „тихи“ catch блок, односно уклонила сам само један позив LogWarning-a.

Резултат је био да тачно један тест падне — управо онај који проверава да неуспешан HTTP статус производи одговарајући лог. Остала два теста из истог пројекта су и даље пролазила.

Након враћања исправке, сви тестови су поново прошли.

СЛАЈД 13 — Демонстрациони сценарији над стварним системом

Оквирно време: 1:20 минута

Поред аутоматизованих тестова, урадила сам и четири демонстрациона сценарија над стварно постављеним системом.

У првом сценарију нормалан захтев пролази кроз три сервиса, а trace приказује чак девет угнежђених SQL child span-ова.

У другом сценарију намерно је угашен TripService. Health check је аутоматски пријавио Unhealthy статус, а систем се опоравио за око седам секунди.

Трећи сценарио је k6 load test са 780 захтева. Клијентска и серверска латенција су биле 2.12 и 2.14 секунди.

У четвртном сценарију намерно је успорена SQL зависност. Trace је омогућио да се узрок локализује на конкретну UPDATE наредбу, уместо да се само види да је цео захтев спор.

СЛАЈД 14 — Дистрибуирано праћење захтева у пракси

Оквирно време: 0:55 минута

Овде је приказан стварни trace из Grafana Tempo-a за први сценарио.

Видимо како захтев иде од Gateway-a, преко SharingService-a, до TripService-a, као и угнежђене SQL операције.

Све ово се дешава аутоматски, без custom кода који би био написан посебно за пропагацију контекста, захваљујући OpenTelemetry инструментацији за ASP.NET Core и HttpClient.

СЛАЈД 15 — Шта је постигнуто

Оквирно време: 1:10 минута

Да сумирам резултате.

Идентификовано је пет конкретних недостатака постојећег система. Затим је имплементирана надоградња заснована на OpenTelemetry и Grafana stack-у, без измене пословне логике.

Резултат је верификован на два начина: кроз 21 аутоматизован тест и кроз четири демонстрациона сценарија над стварно постављеним системом.

Као могући правци даљег развоја издвојила бих формално alerting решење, инструментацију frontend-a за потпуни end-to-end trace и аутоматизацију k6 load теста у оквиру CI/CD pipeline-a.

Тиме је испуњен основни циљ рада — да се не покаже само да су алати инсталирани, већ да надоградња заиста даје употребљив увид у понашање система.

СЛАЈД 16 — Хвала на пажњи / Питања

Оквирно време: 0:05 минута

Хвала на пажњи. Ту сам за евентуална питања.